PROJECT DOCUMENTATION

Networking Issues in Mobile Systems. Part 1: Conceptual Design

Project topic: Mobile Currency Exchange System

1. General Information

Project Name	Currency Exchange
Project Authors	Oleh Smolinskyi
Field of Study	Computer Science
Year / Semester	4 / 7
Supervisor	Krzysztof Rychlicki-Kicior
Submission Date	01.11.2026
Repository	https://github.com/olezhakashz/currency-exchange, tag `part1`

2. Project Description

2.1. Project Objective

The goal of the project is a mobile app for virtual currency exchange. The user can see current exchange rates from the National Bank of Poland (NBP), convert amounts, and buy or sell currency from a virtual wallet that starts with 1000 PLN. No real money is used.

The main point of the project is not the number of features but how the app behaves when the wireless connection is bad: rates are cached and labelled with their date, orders are blocked when the phone is offline, and if the reply to an order is lost the app can still find out what really happened.

2.2. Project Scope

The system has four parts:

- **Mobile app**: shows rates, converts amounts, shows the wallet, sends buy and sell orders. It keeps a local cache of the last rates and wallet, and a list of orders that are still waiting for an answer.
- **Web service**: a REST API over HTTPS. It logs users in, gets rates from NBP, checks and executes orders, and answers status questions about orders.
- **Database**: users, balances, orders and a dated copy of the NBP rates.
- **NBP API**: external system. It owns the exchange rates. My server only keeps a copy.

I kept the scope small on purpose. I left out rate history, topping up the wallet and a transaction history screen because they are only extra reads and don't add anything new on the network side (see section 8).

Technology: Flutter for the app, Python with FastAPI for the web service, SQLite for the database, Git and GitHub for version control.

3. System Requirements

3.1. Functional Requirements

ID	Function Name	Description	Priority
F1	Register / log in	The user can create an account with email and password and log in. The server returns a token.	High
F2	View current rates	The app shows NBP table A mid rates, each with its effective date.	High
F3	Convert amount	The user can convert between PLN and a chosen currency with the displayed rate. Done on the phone.	Medium
F4	View wallet	The user can see balances. A new user starts with 1000.00 PLN.	High
F5	Buy currency	The user can buy a currency with PLN at the current server rate.	High
F6	Sell currency	The user can sell a currency for PLN at the current server rate.	High
F7	Reject low balance	The server rejects a buy or sell if there is not enough money.	High
F8	Request ID	Every order gets a unique request ID on the phone before it is sent.	High
F9	Order status	The app can ask the server for the status of an order by its request ID.	High
F10	Reject changed rate	The server rejects an order if the rate the user saw is no longer the current rate, and sends the new rate.	Medium

3.2. Non-Functional Requirements

ID	Name	Description	Category
N1	Timeout	Every request from the app has a 10 s timeout.	Performance
N2	Cached label	Cached data is shown with its date and a "cached" label.	Usability
N3	Stale rates	Rates older than the latest NBP publication day are marked "stale".	Data correctness

ID	Name	Description	Category
N4	Offline blocking	When offline, buy and sell are disabled within 1 s, with a message.	Reliability
N5	At most once	The server runs an order at most once per request ID.	Reliability
N6	Retries	Reads are retried up to 3 times (1 s, 2 s, 4 s). Orders are never resent automatically; the app checks the status instead.	Reliability
N7	Reconnection	After reconnecting, the app refreshes rates and wallet within 5 s and resolves orders with unknown result.	Reliability
N8	Atomic balances	Balance changes are done in one database transaction; a balance can never go below zero.	Data correctness
N9	NBP calls	The server calls NBP at most once per hour and serves its saved copy otherwise.	Performance
N10	Security	All traffic uses HTTPS; wallet and order endpoints need a valid token.	Security

3.3. Network-Failure Scenarios

Offline policy: reads can be shown from the cache, actions (buy, sell) are blocked while offline, because only the server can confirm an order at the current rate and balance.

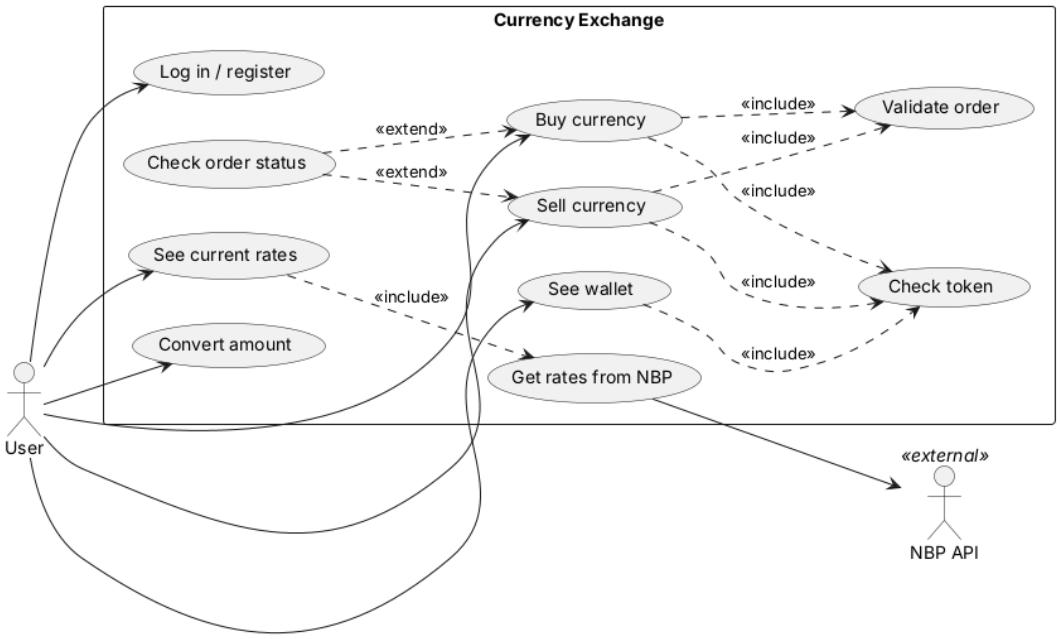
ID	Scenario	What the user sees	Server state	Expected result (test)
S1	Offline before sending	Buy button disabled, “You’re offline”	Nothing received	No request sent, no order in DB (N4)
S2	Reply lost after the order ran	“Checking order…”	Order saved as CONFIRMED	After reconnect the app asks for status and shows “Confirmed”; exactly 1 order (F9, N7)
S3	Request lost before the server	“Checking order…”	Nothing saved	Status is NOT_FOUND; app shows “Not executed, try again”
S4	Timeout	After 10 s: cached data for reads, “Checking order…” for orders	Unknown until checked	The app never shows success without server confirmation (N1)
S5	Slow reply (8 s)	Spinner, then the result	Normal	Result shown once, no second request
S6	Read fails twice	Spinner, then rates	Unchanged	Third try works; max 3 tries (N6)
S7	Same order sent	One “Confirmed”	One order	Server returns the saved

ID	Scenario	What the user sees	Server state	Expected result (test)
	twice			result for the repeat (F8, N5)
S8	App opened offline	Cached rates with date, wallet “last known”, buttons disabled	Unchanged	Labels visible, no action possible (N2, N4)
S9	Stale rates	Rates marked “stale”	Old rate copy	Order with an old rate is rejected with the new rate (N3, F10)
S10	Reconnection	Labels disappear, fresh data	Unchanged	Within 5 s the app matches the server (N7)
S11	Two devices sell all EUR at once	One “Confirmed”, one “Not enough money”	One sell saved	Exactly one succeeds, balance never negative (N8, F7)
S12	Rate changes before submit	“Rate changed to X. Confirm?”	No order	Nothing runs until the user confirms (F10)
S13	Double tap on Buy	One “Checking…”, then “Confirmed”	One order	Button disabled after first tap (F8)
S14	NBP is down	Rates marked “stale”	Last saved rates	App keeps working with labelled rates

4. UML Diagrams

The diagrams are made in PlantUML. The editable source files (`.puml`) are in the repository in `docs/diagrams/`.

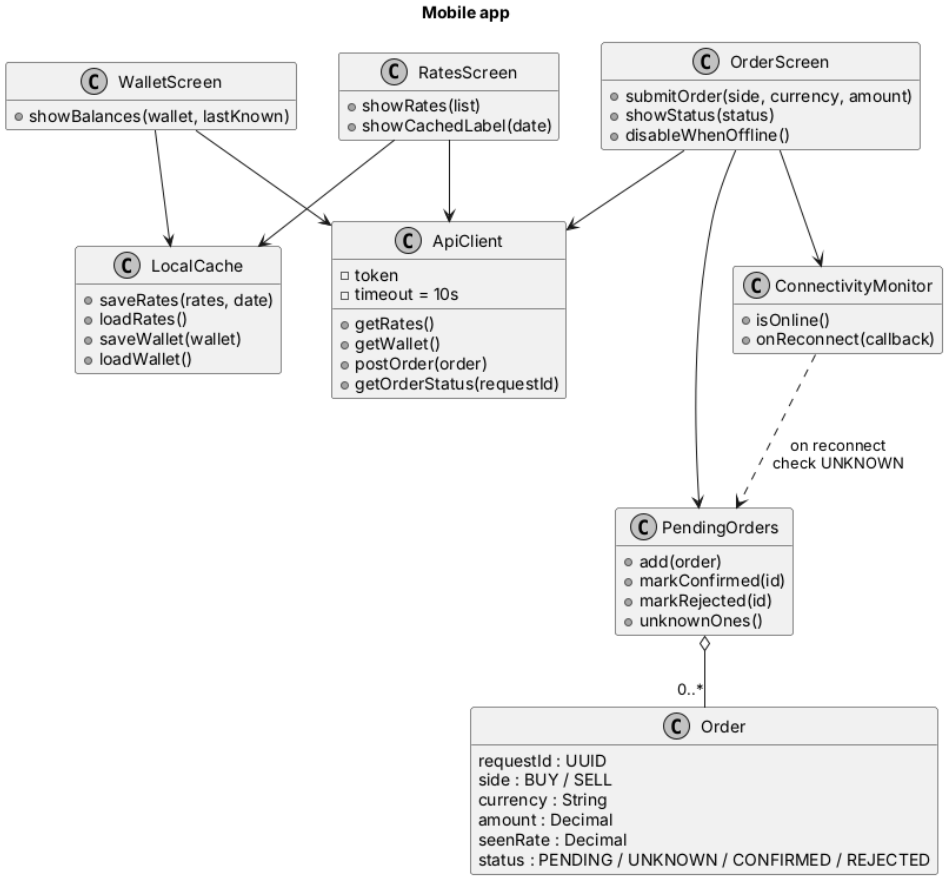
4.1. Use Case Diagram



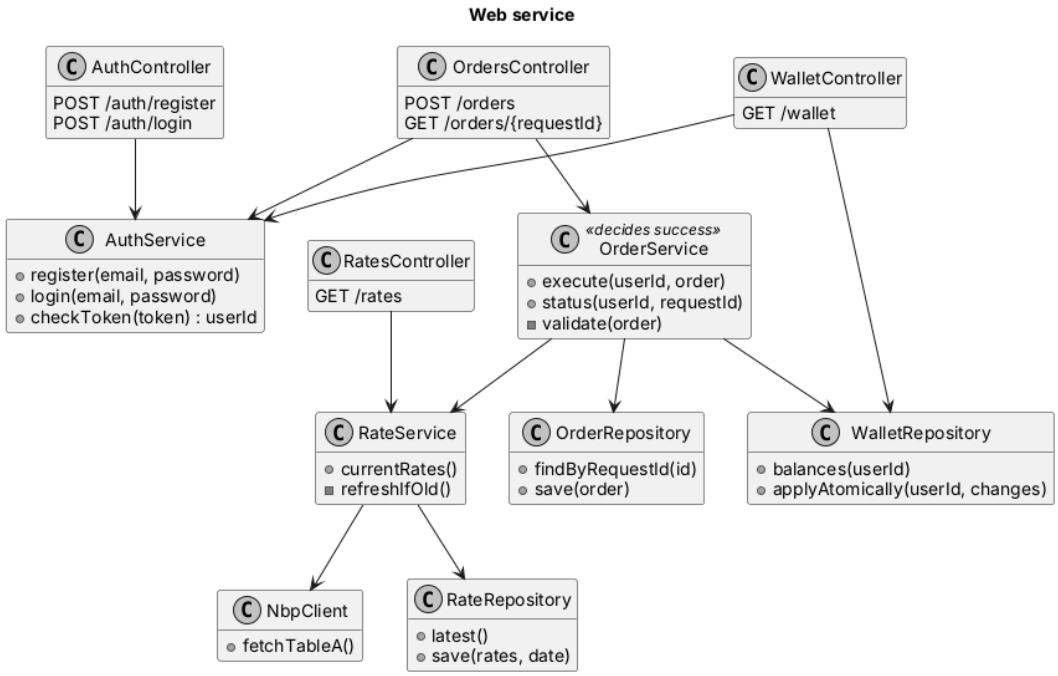
Use-case diagram

There are two actors: the user, and the NBP API as an external system. Every use case that touches the wallet includes checking the token. Buy and sell also include server-side validation. “Check order status” extends buy and sell: it only happens when the answer to an order didn’t arrive.

4.2. Class Diagram



Mobile app classes

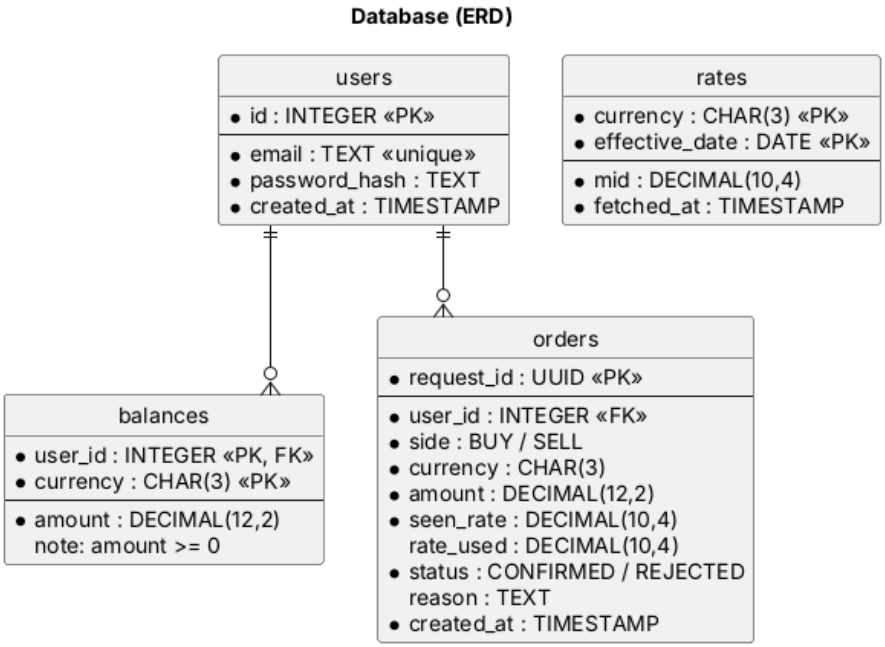


Web service classes

Responsibilities. On the phone, the screens use `ApiClient` (with the 10 s timeout) to talk to the server, `LocalCache` to show data offline, `ConnectivityMonitor` to block buttons when offline, and `PendingOrders` to remember orders that have no answer yet. On the server, `AuthService` checks tokens, `RateService` serves rates and calls NBP through `NbpClient` only when the saved copy is old, and `OrderService` validates and executes orders.

`OrderService` is the only component that decides whether an order succeeded. Its decision is the row in the `orders` table. The phone only shows what the server confirmed.

5. Database Design

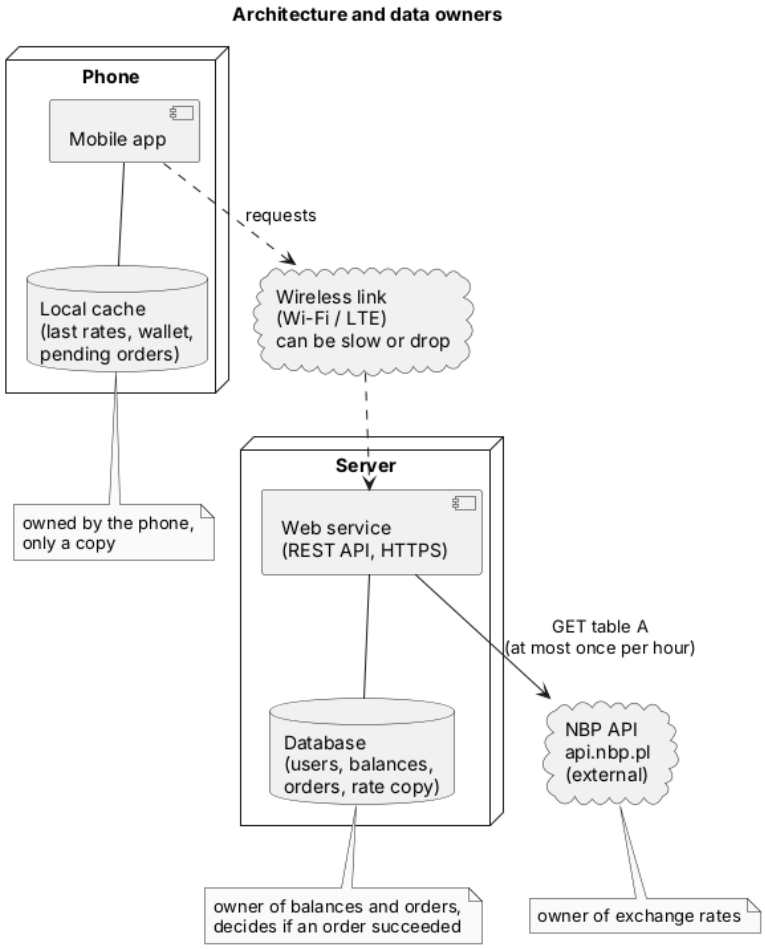


ERD

Table	Primary key	Foreign keys	What it stores
users	id (INTEGER)	none	email (unique), password hash, created at
balances	user_id + currency	user_id → users.id	amount (DECIMAL(12,2), must be ≥ 0) per currency
orders	request_id (UUID)	user_id → users.id	side (BUY/SELL), currency, amount, rate the user saw, rate used, status, reason, time
rates	currency + effective_date	none	NBP mid rate (DECIMAL(10,4)) and when it was fetched

One user has many balances (one per currency) and many orders. `request_id` is the primary key of `orders`, so the database itself stops the same order from being saved twice. The `rates` table has no link to users because rates are the same for everyone and belong to NBP.

6. System Architecture



Architecture

The phone talks only to my web service, over the wireless link, which can be slow or drop. The web service talks to the database and, at most once per hour, to NBP.

Data owners: NBP owns the exchange rates. The web service and its database own users, balances and orders. The phone only has copies (cache) and a list of its own pending orders.

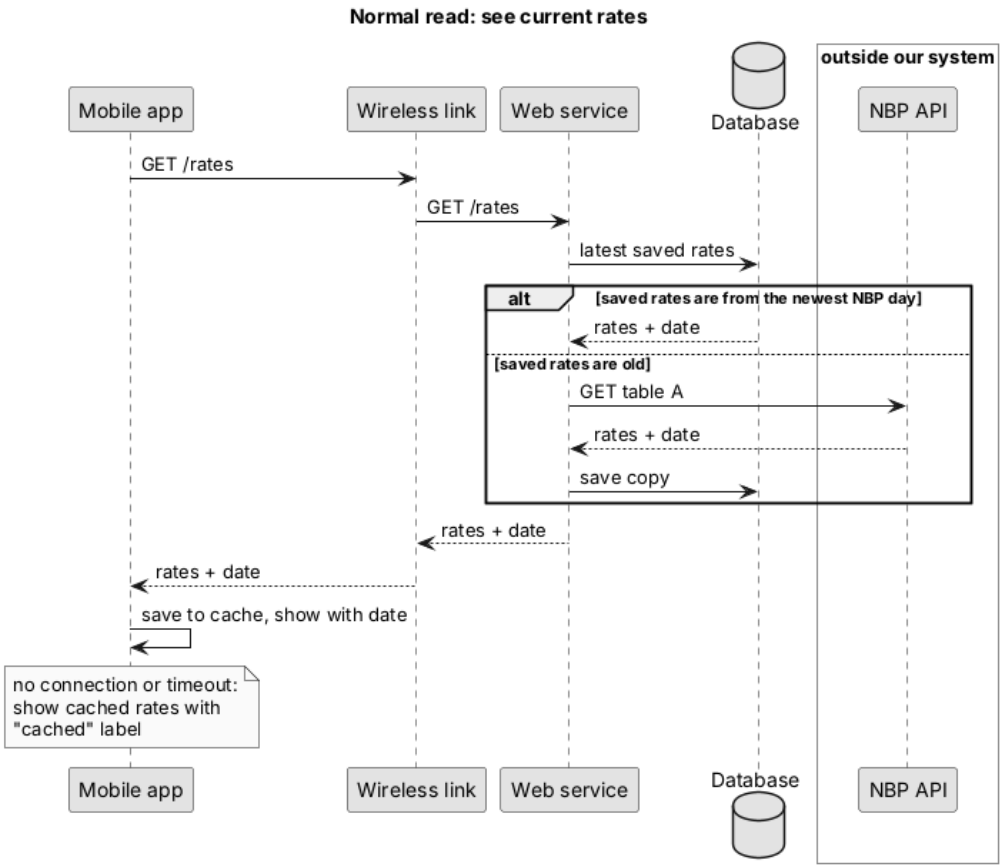
6.1. API Outline

Method	Path	Token	Type	Purpose
POST	/auth/register	no	action	create a user with 1000 PLN
POST	/auth/login	no	action	get a token
GET	/rates	no	read	current rates, date, stale flag
GET	/wallet	yes	read	balances
POST	/orders	yes	action	buy or sell, at most once per requestId
GET	/orders/{requestId}	yes	read	CONFIRMED, REJECTED or NOT_FOUND

Full request and response examples are in `docs/api.md` in the repository.

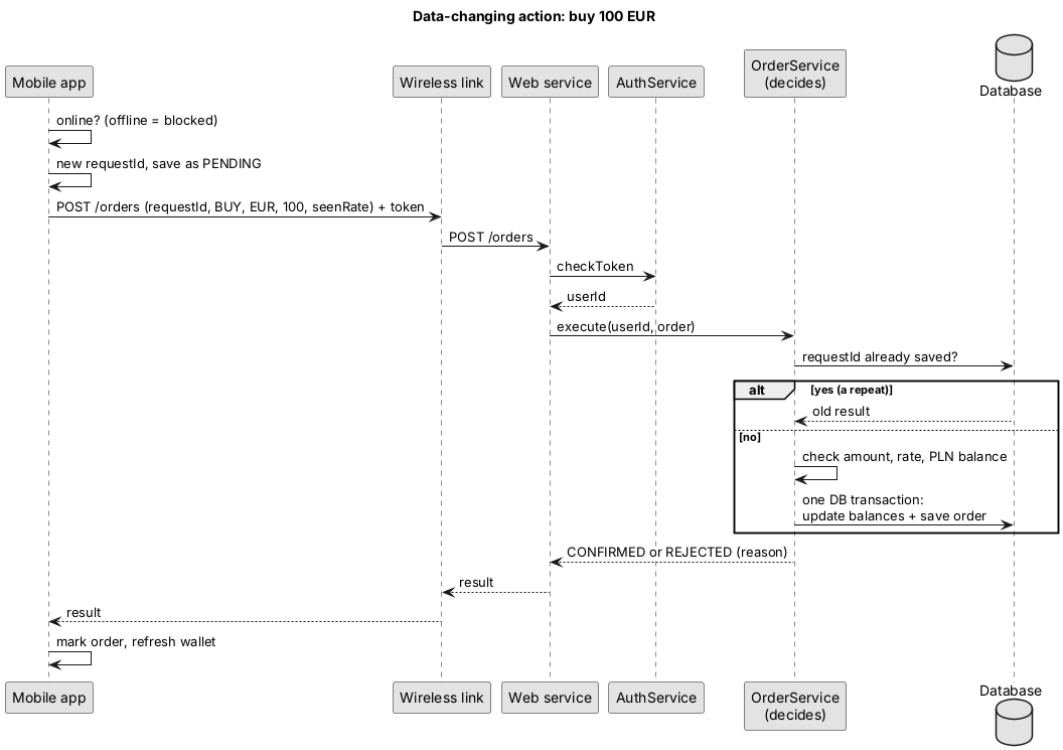
6.2. Data Flow

Normal read (see rates).



Read flow

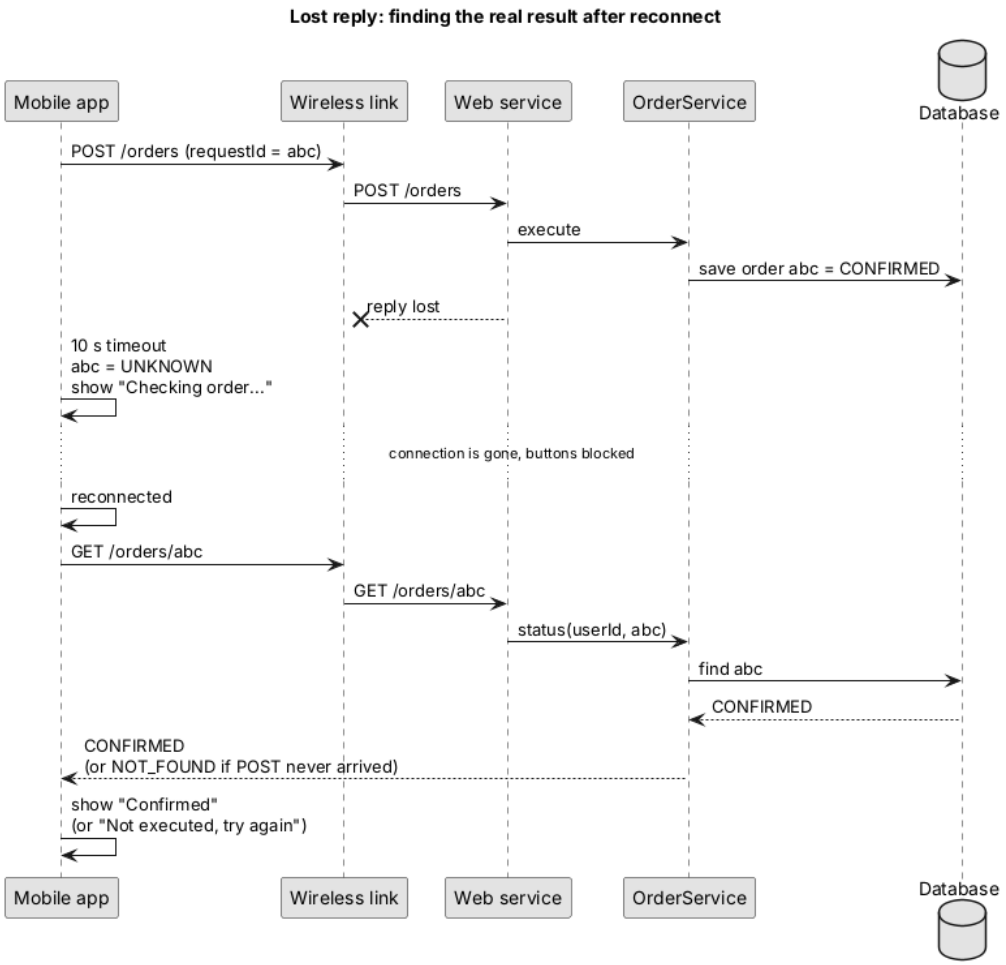
Data-changing action (buy 100 EUR).



Buy flow

When I press Buy, the app first checks that I'm online. Then it gives the order a random request ID and saves it on the phone as PENDING. The order is sent with the rate I saw and my token. The server checks the token, then checks whether this request ID already exists. If it does, it just returns the old answer. If not, it checks the amount, the rate and my PLN balance, and then changes both balances and saves the order in one database transaction. The server answers CONFIRMED or REJECTED and the phone shows that.

Lost reply (finding the real result after reconnecting).



Recovery flow

7. Project Implementation Plan

Stage	Description	Deadline	Responsible Person
1	Requirements analysis and network scenarios	Done (Lab 02)	Oleh
2	UML diagrams and data flow	Done (Lab 03)	Oleh
3	Part 1 design report, ERD, API outline	Lab 04	Oleh
4	Web service: rates endpoint and NBP client with caching	[date]	Oleh
5	Login, wallet, database	[date]	Oleh
6	Orders: buy, sell, request ID, status endpoint	[date]	Oleh
7	Mobile app screens and local cache	[date]	Oleh

Stage	Description	Deadline	Responsible Person
8	Offline blocking, retries, reconnection handling	[date]	Oleh
9	Testing the network scenarios S1 to S14	[date]	Oleh
10	Part 2 report and demo	[date]	Oleh

8. Conclusions and Possible Extensions

The design puts all decisions about money on the server and treats the phone's data as copies. That makes the bad-network cases manageable: the phone can always show something (with a label), and it can always find out the real result of an order by asking with its request ID.

Possible extensions after the main project is done:

- rate history chart for one currency (NBP has an endpoint for it)
- transaction history screen
- topping up the wallet
- push notification when a rate reaches a value the user chose
- queueing orders while offline, with the user confirming again when the phone is back online
- more currencies (NBP table B) and buy/sell rates (table C)

9. Sources

1. National Bank of Poland, Web API documentation: https://api.nbp.pl/en.html
2. FastAPI documentation: https://fastapi.tiangolo.com
3. Flutter documentation: https://docs.flutter.dev
4. SQLite documentation: https://www.sqlite.org/docs.html
5. PlantUML language reference: https://plantuml.com
6. RFC 9110, HTTP Semantics (idempotent methods), IETF, 2022: https://www.rfc-editor.org/rfc/rfc9110
7. Lecture materials, Networking Issues in Mobile Systems